



نظري

5

28

3360 sp

كلية الهندسة المعلوماتية

السنة الثالثة

التحويل من نموذج الكيانات والارتباطات إلى النموذج العلاقتي

د. سعيد دسوقي

محتوى مجاني غير مخصص للبيع التجاري

RB Informatics; 07/05/2026

قواعد معطيات

إن مخطط الكيانات والارتباطات (Entity-Relationship Model) الذي تحدثنا عنه سابقاً يُستخدم في مرحلة التصميم والتحليل فقط، لأنه يوضّح الكيانات والعلاقات بينها بشكل مفاهيمي، لكنه ليس نموذجاً تنفيذياً يمكن تطبيقه مباشرة داخل نظام إدارة قواعد المعطيات. لذلك نقوم بتحويله إلى النموذج العلاقتي (Relational Model)، لأن النموذج العلاقتي يعتمد على الجداول ويمكن تنفيذه فعلياً داخل قواعد المعطيات.

تحويل الكيانات القوية ذات الواصفات البسيطة

- يتحول الكيان القوي ذو الواصفات البسيطة إلى علاقة لها الواصفات نفسها والمفتاح الرئيسي نفسه :

course (course_id, title, credit)

course
<u>course_id</u>
title
credits

تحويل الكيانات الضعيفة

- يتحول الكيان الضعيف إلى علاقة لها الواصفات نفسها إضافةً إلى واصفات المفتاح الرئيسي للكيان القوي المحدد له التي تصبح مع الواصفات المميزة مفتاحاً رئيسياً للعلاقة، وتكون وحدها مفتاحاً خارجياً يمثل الارتباط بين الكيانين:

section (course_id, sec_id, semester, year)



مثال على الكيان الضعيف:

الكيان section يُعتبر كياناً ضعيفاً لأنه يعتمد على الكيان القوي course، لذلك يتحول إلى العلاقة:
 . section(course_id, sec_id, semester, year)

هنا يكون course_id جزءاً من المفتاح الأساسي، وفي الوقت نفسه مفتاحاً خارجياً (Foreign Key) يشير إلى العلاقة course.

أما المفتاح الأساسي الكامل للعلاقة فيتكوّن من: course_id و sec_id و semester و year معاً.

تحويل الأنواع المختلفة من الواصفات

instructor
<u>ID</u>
name
first_name
middle_initial
last_name
address
street
street_number
street_name
apt_number
city
state
zip
{ phone_number }
date_of_birth
age ()

1. الواصفة المركبة (Composite Attribute):

عند تحويل الواصفة المركبة، لا نقوم بوضعها كما هي، بل نأخذ الواصفات الموجودة في المستوى الأخير فقط.

مثال:

إذا كانت الواصفة name تتكوّن من : first_name - last_name

فإننا نحولها إلى: name_first - name_last

وذلك للحفاظ على العلاقة بين الواصفات ومعرفة أنها تنتمي لنفس الواصفة المركبة.

2. الواصفة متعددة القيم (Multivalued Attribute):

لا يمكن تمثيل الواصفة متعددة القيم داخل نفس الجدول، لذلك يتم إنشاء جدول مستقل لها.

مثال:

إذا كان المدرّس Instructor يمتلك عدة أرقام هاتف، نقوم بإنشاء جدول خاص مثل:

instructor_phone(ID, phone)

بحيث يرتبط رقم الهاتف بالمدرّس عن طريق الـ ID الخاص به.

الـ primary key هنا هو رقم الهاتف، لأن الـ ID سيكرر).

3. الوصفة المشتقة (Derived Attribute):

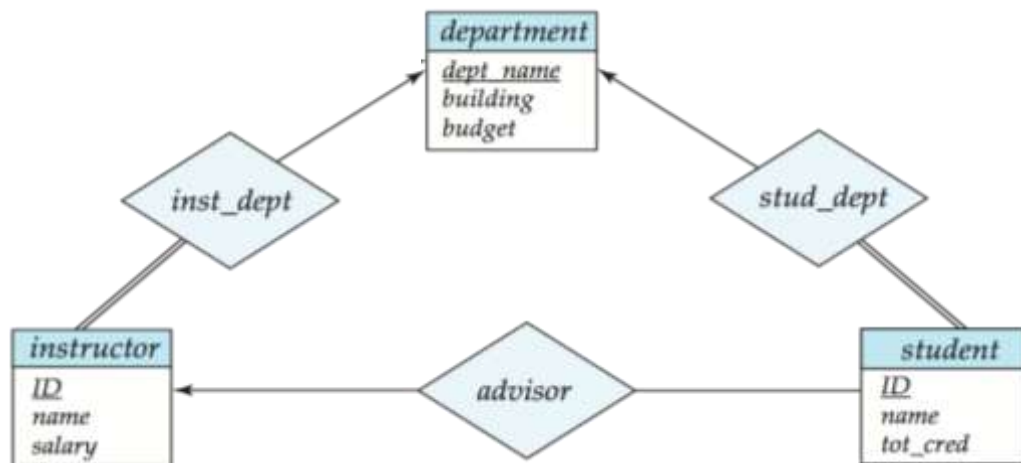
الوصفة المشتقة لا يتم تخزينها داخل قاعدة المعطيات، لأنها تحسب انطلاقاً من واصفات أخرى.

مثال:

يتم تخزين: birthdate بينما: age لا يتم تخزينه لأنه يمكن حسابه اعتماداً على تاريخ الميلاد.

الارتباطات واحد لعدة

- يجري تحويل الارتباط one-to-many بإضافة المفتاح الرئيسي للعلاقة الناتجة عن الكيان الموجود في الطرف one إلى العلاقة الناتجة عن الكيان الموجود في الطرف many ليصبح مفتاحاً خارجياً فيها:



student (ID, name, tot_cred)

instructor (ID, name, salary)

- يمكن للقسم Department أن يحتوي على عدة طلاب وعدة مدرّسين، بينما ينتمي كل طالب Student أو مدرّس Instructor إلى قسم واحد فقط، لذلك نقوم بإضافة الوصفة dept_name إلى علاقتي student و instructor لتصبح بالشكل:

student(ID, name, tot_cred, dept_name)

instructor(ID, name, salary, dept_name)

- وهنا نعتبر dept_name مفتاحاً خارجياً (Foreign Key) يشير إلى العلاقة department(dept_name).

إذا كانت الارتباط اختيارياً من الطرف many فمن الأفضل إنشاء علاقة جديدة تعبر عن الربط:

المثال:

العلاقة advisor بين: student و instructor تمثل إشراف المدرّس على الطالب.

ولأن الإشراف قد يكون اختيارياً، يمكن تمثيله بعلاقة مستقلة:

advisor(S_ID, I_ID)

بحيث: S_ID يشير إلى الطالب ، I_ID يشير إلى المدرّس

الارتباطات واحد لواحد

- في علاقات One-to-One يمكن إضافة المفتاح الأساسي Primary Key لأحد الكيانيين إلى العلاقة الناتجة عن الكيان الآخر ليصبح مفتاحاً خارجياً Foreign Key
- من الأفضل إضافة المفتاح الخارجي إلى علاقة الطرف الذي يكون فيه الارتباط إجبارياً لأن ذلك يقلل من ظهور القيم NULL داخل الجدول.

مثال اربيسيبي:

إذا كان كل Student يجب أن يمتلك Advisor، بينما ليس بالضرورة أن يشرف كل Instructor على طالب، فمن الأفضل إضافة الوصفة instructor_id إلى العلاقة student بدل إضافتها إلى العلاقة instructor، وذلك لأن جميع الطلاب سيملكون قيمة فعلية لهذا الحقل، بينما لو أُضيف إلى instructor فستظهر العديد من القيم NULL للمدرّسين الذين لا يشرفون على طلاب.

إذا كان الارتباط اختيارياً من الطرفين فمن الأفضل إضافة المفتاح الخارجي إلى علاقة الطرف الذي يتوقع أن تكون عدد عناصره أقل. لأن الجدول ذو العناصر الأقل سيحتوي عدداً أقل من القيم الفارغة.

مثال:

إذا كان لدينا 1000 طالب و100 مدرس، وقمنا بإضافة student_id داخل جدول instructor فسنحتاج إلى 100 قيمة فقط. أما إذا أضفنا instructor_id داخل جدول student فسيصبح لدينا 1000 حقل، وقد يحتوي عدد كبير منها على القيمة NULL. لذلك نفضّل إضافة المفتاح الخارجي إلى الجدول الذي يحتوي عدد عناصر أقل، وذلك لتقليل القيم الفارغة وتحسين التخزين والأداء.



الارتباطات عدة لعدة Many-to-Many

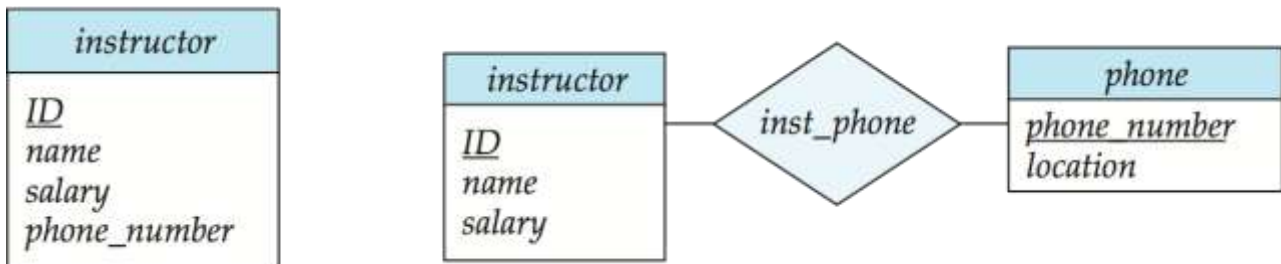
- عند تحويل علاقة Many-to-Many إلى النموذج العلائقي، لا يمكن تمثيلها بإضافة مفتاح خارجي فقط، لذلك يتم إنشاء علاقة جديدة مستقلة تمثل هذا الارتباط. تحتوي العلاقة الجديدة على المفاتيح الأساسية للكيانات المشاركة، وتشكل هذه المفاتيح معاً المفتاح الأساسي Primary Key للعلاقة الجديدة، كما يُعتبر كل واحد منها مفتاحاً خارجياً Foreign Key يشير إلى الكيان الذي أتى منه .
- إذا احتوت العلاقة على واصفات إضافية، فإن هذه الواصفات تُضاف إلى العلاقة الجديدة الناتجة عن التحويل

advisor (s_id, i_id)

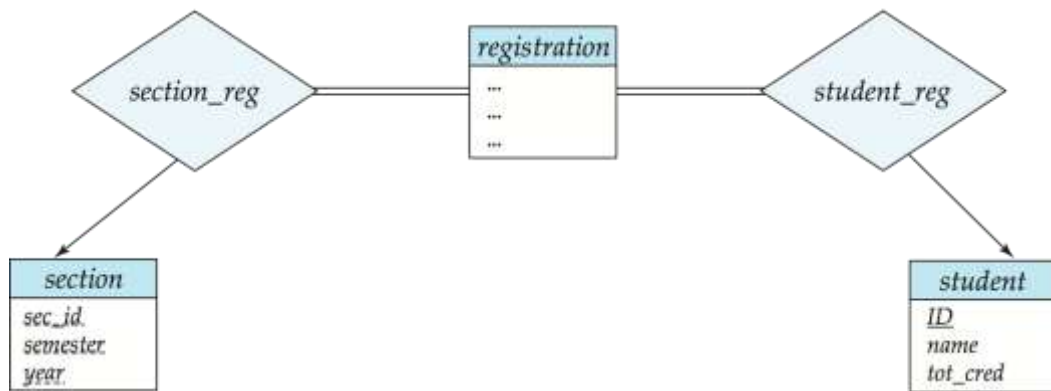


ملاحظات تصميمية

كيان أم واصفة؟



- يمكن تمثيل رقم الهاتف على أنه واصفة Attribute داخل الكيان instructor:
instructor(ID, name, salary, phone_number)
- وهذا مناسب إذا كان المدرّس يملك رقم هاتف واحد فقط، ولا توجد معلومات إضافية مرتبطة به.
- لكن إذا كان رقم الهاتف يحتاج معلومات أخرى مثل:
Location - عدة أرقام
- فمن الأفضل تمثيله ككيان مستقل phone مرتبط بالمدرّس بعلاقة inst_phone .
- ✎ أي أن القرار يعتمد على طبيعة البيانات وتعقيدها.

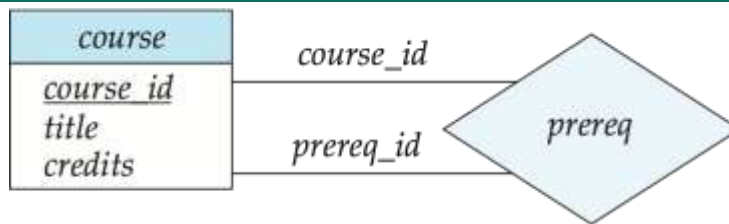


- تم تمثيل registration ككيان مستقل بدل أن تكون مجرد علاقة بين section و student .

🔗 نلجأ لهذا الحل عندما تحتوي العلاقة على معلومات خاصة بها

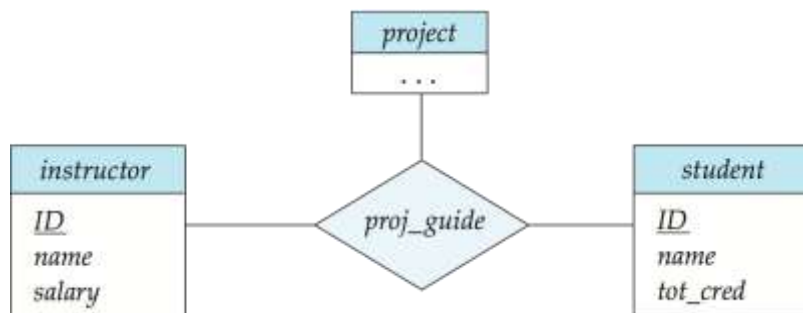
- فبدل: student ↔ section
- أصبحت: student ↔ registration ↔ section

الأدوار



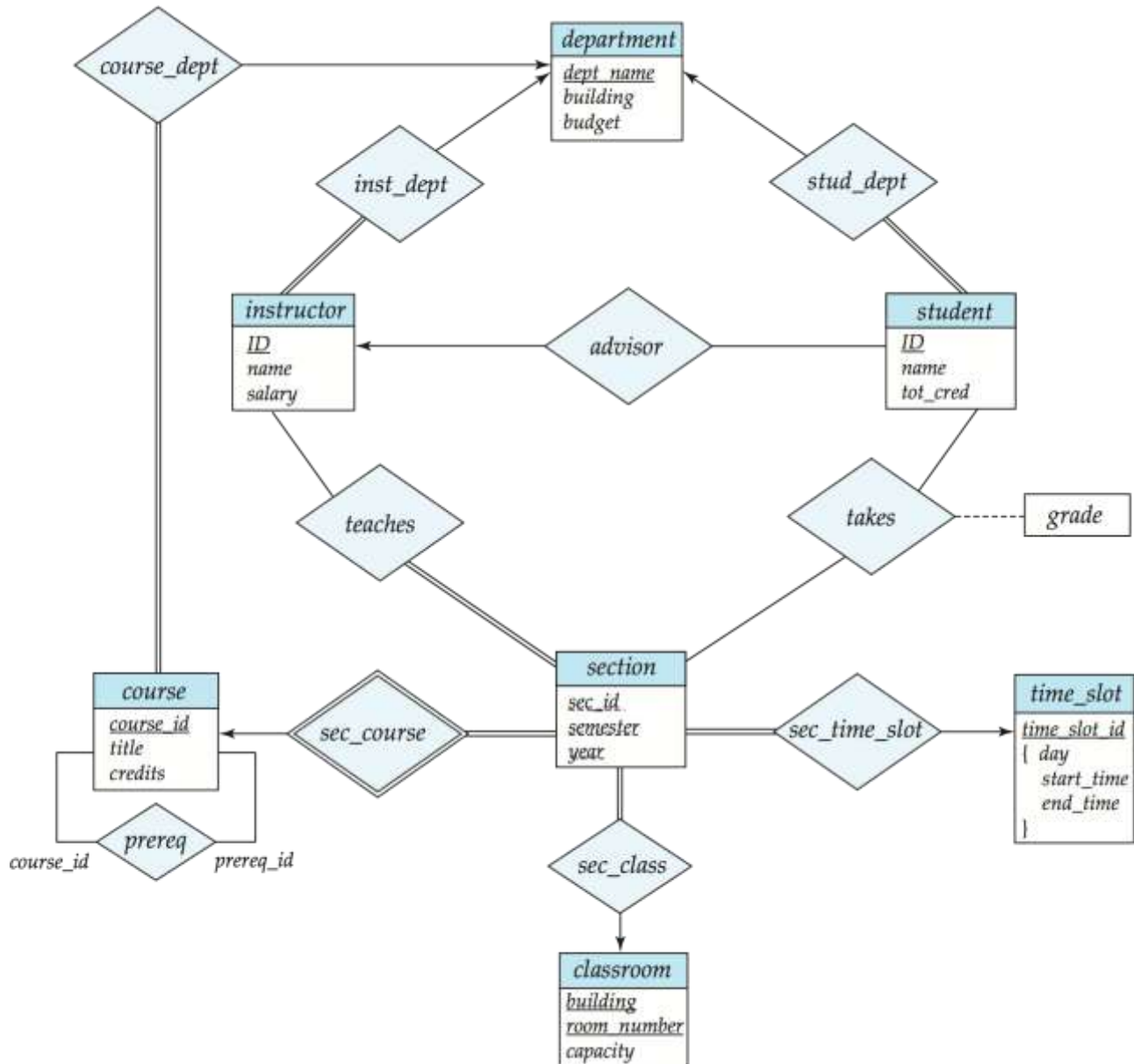
- في العلاقة الذاتية Recursive Relationship الخاصة بال prereq، يُعتبر كل من **course_id** و **prereq_id** مفتاحين أساسيين Primary Key معاً، وذلك لأن العلاقة من النوع Many-to-Many.
- وفي الوقت نفسه، يُعدّ كل واحد منهما بشكل منفصل مفتاحاً خارجياً Foreign Key يشير إلى العلاقة **course** نفسها، لأن العلاقة ناتجة عن ارتباط الكيان **course** مع نفسه.
- تُكتب أسماء الأدوار على طرفي العلاقة، وينتج عن التحويل جدول العلاقة prereq إلى جانب جدول الكيان الأصلي **course**.

العلاقات الثلاثية:



- أما في العلاقات الثلاثية (Ternary Relationships)، فيتم إنشاء علاقة جديدة تحتوي على المفاتيح الأساسية للكيانات الثلاثة المشاركة، وتشكّل هذه المفاتيح معاً المفتاح الأساسي للعلاقة الجديدة. وينطبق المبدأ نفسه على العلاقات الرباعية أو أي علاقات متعددة الأطراف.

قم بتحويل مخطط الكيانات والارتباطات الخاص بالجامعة إلى النموذج العلاقتي (Relational Schema).



في رحلة العُمُر والأيام مُسرعةً
لا تنسَ مَنْ أنت أو ما وجهةُ السَّفَرِ 🌟 🕊



يُستخدم التخصيص (Specialization) والتعميم (Generalization) لتمثيل العلاقات الهرمية بين الكيانات.

في التخصيص (Specialization) نبدأ من كيان عام ثم نقسمه إلى كيانات أكثر تخصصًا، لذلك يُسمّى أسلوب Top-Down.

أما في التعميم (Generalization) فنبدأ من كيانات متخصصة ثم نجمعها ضمن كيان عام، لذلك يُسمّى Bottom-Up.

■ علاقة IS-A

تُسمَّى العلاقة بين الكيان العام والكيانات المتخصصة علاقة وراثثة (Inheritance) أو علاقة IS-A. حيث يُسمَّى الكيان الأعلى Superclass، بينما تُسمَّى الكيانات المتفرعة عنه Subclasses. مثلاً: (student is-a person)

■ الوراثة (Inheritance)

في هذا النوع من العلاقات، لا ترث الكيانات المتخصصة (Subclasses) الواصفات (Attributes) فقط، بل ترث أيضاً العلاقات (Relationships) المرتبطة بالكيان العام في حال وجودها.

■ مثال

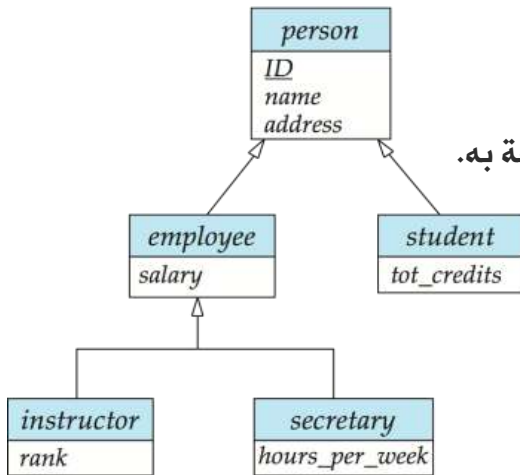
الكيان Person يُعدّ كياناً عاماً، بينما Student و Employee يُعتبران كيانات متخصصة ترث صفاته الأساسية مثل:

ID - name - address

كما يمكن أن يتفرع الكيان Employee إلى كيانات أكثر تخصصاً مثل:

Instructor - Secretary

بحيث يرث كل كيان الصفات الخاصة بالكيان الأعلى، ويضيف صفاته الخاصة به.



◇ ملاحظة :

يمكن أن توجد عدة تخصيصات أو تعميمات للكيان نفسه بحسب التصنيف المطلوب.

✍ مثال:

permanent_employee / temporary_employee -

Instructor / Secretary -



قيود التخصيص والتعميم

تستخدم قيود التخصيص والتعميم لتحديد كيفية انتماء العناصر ضمن الهرمية بين الكيانات.

تحديد الانتماء إلى الكيانات الفرعية:

يمكن تحديد انتماء العنصر إلى كيان فرعي بطريقتين:

1. اعتماداً على شرط Condition-Defined

يتم تحديد نوع الكيان اعتماداً على قيمة إحدى الواصفات (Properties)

✍ مثال:

إذا كان عمر الشخص أكبر من 65 سنة يتم تصنيفه ضمن senior-citizen، أما إذا كان أصغر من ذلك فيبقى ضمن الفئة العادية.

2. اعتماداً على اختيار المستخدم

يتم تحديد نوع الكيان أثناء إنشاء العنصر أو إدخال البيانات من قبل المستخدم.

هل يمكن أن ينتمي العنصر إلى أكثر من كيان على نفس المستوى؟

نعم → { Overlapping تخصيص أو تعميم تقاطعي }

يمكن للعنصر أن ينتمي إلى أكثر من كيان فرعي في الوقت نفسه.

مثال: قد يكون الشخص Student و Employee معاً، فهناك تقاطع بين مجموعتي الموظفين والطلاب.

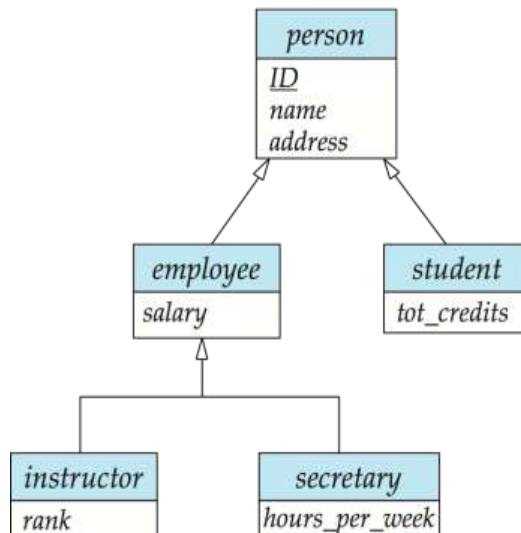
لا → { Disjoint تخصيص أو تعميم انفصالي }

لا يمكن للعنصر أن ينتمي إلا إلى كيان فرعي واحد فقط على نفس المستوى

من ناحية التمثيل في الرسم:

- في حالة الـ Disjoint تكون الكيانات الفرعية على نفس السطر، لأن العنصر لا يمكن أن ينتمي إلا إلى كيان واحد فقط.

- أما في حالة الـ Overlapping ترسم الكيانات الفرعية على أكثر من سطر، لأن العنصر يمكن أن ينتمي إلى عدة كيانات في الوقت نفسه.



قيد التكامل Completeness Constraint:

- يحدد فيما إذا كان كل عنصر في الكيان الأعلى يجب أن ينتمي إلى أحد الكيانات الفرعية أم لا.

{Total تخصيص أو تعميم كامل} :

- يجب أن يُصنف كل عنصر ضمن أحد الكيانات الفرعية.

✍ مثال:

كل Person يجب أن يكون إما Student أو Employee، ولا يوجد عنصر يبقى ضمن Person فقط.

{Partial تخصيص أو تعميم جزئي} :

- ليس من الضروري أن ينتمي كل عنصر إلى كيان فرعي.

✍ أي يمكن أن يوجد Person لا يكون Student ولا Employee، بل يبقى ضمن الكيان العام فقط.

تحويل التخصيص والتعميم إلى النموذج العلاقتي

- يمكن تمثيل التخصيص والتعميم في النموذج العلاقتي بعدة طرق:

▪ الطريقة الأولى: علاقة واحدة :

يتم إنشاء علاقة واحدة تحتوي على جميع الواصفات الخاصة بالكيان العام والكيانات الفرعية معاً.

• مثال: Person(ID, name, address, salary, tot_credits, rank, hours_per_week)

في هذه الحالة، لن يستخدم كل شخص جميع الحقول، فمثلاً الطالب لا يملك salary، والموظف لا يملك tot_credits، لذلك ستبقى العديد من الحقول بقيمة NULL، مما يؤدي إلى وجود عدد كبير من القيم الفارغة داخل الجدول، فالأفضل أن تُقسم إلى أكثر من علاقة.



الطريقة الثانية: عدة علاقات :

بحيث يتم إنشاء علاقة للكيان العام، ثم علاقات مستقلة للكيانات الفرعية.

مثال: Employee(ID, salary) - Student(ID, tot_credits) - Person(ID, name, address) :
في هذه الطريقة، يكون الـ ID الموجود في Student أو Employee مفتاحاً خارجياً (Foreign Key) يشير إلى Person(ID).

تفيد هذه الطريقة في علاقات التخصيص الجزئي (Partial).

سلبية هذه الطريقة:

عند الحاجة للحصول على معلومات كاملة عن الطالب أو الموظف، يجب تنفيذ عملية Join بين الجداول، مثل:

Student JOIN Person

وعمليات الـ Join تعتبر مكلفة من ناحية الأداء، لأنها تتطلب البحث والمقارنة بين أكثر من جدول (هنا عبارة عن عمليتي for متداخلة $\Leftarrow$ تعقيد n^2).

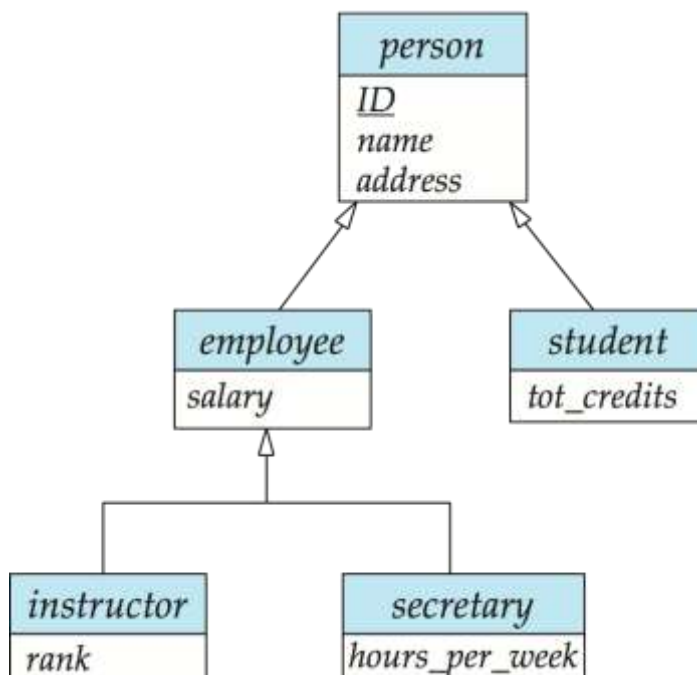
الطريقة الثالثة: عدة علاقات

بحيث يتم تكرار الواصفات العامة داخل كل كيان فرعي.

مثال: Employee(ID, name, address, salary) - Student(ID, name, address, tot_credits) :
في هذه الحالة يتم تجنب عمليات الـ Join، وبالتالي يتحسن الأداء وسرعة القراءة.

سلبية هذه الطريقة:

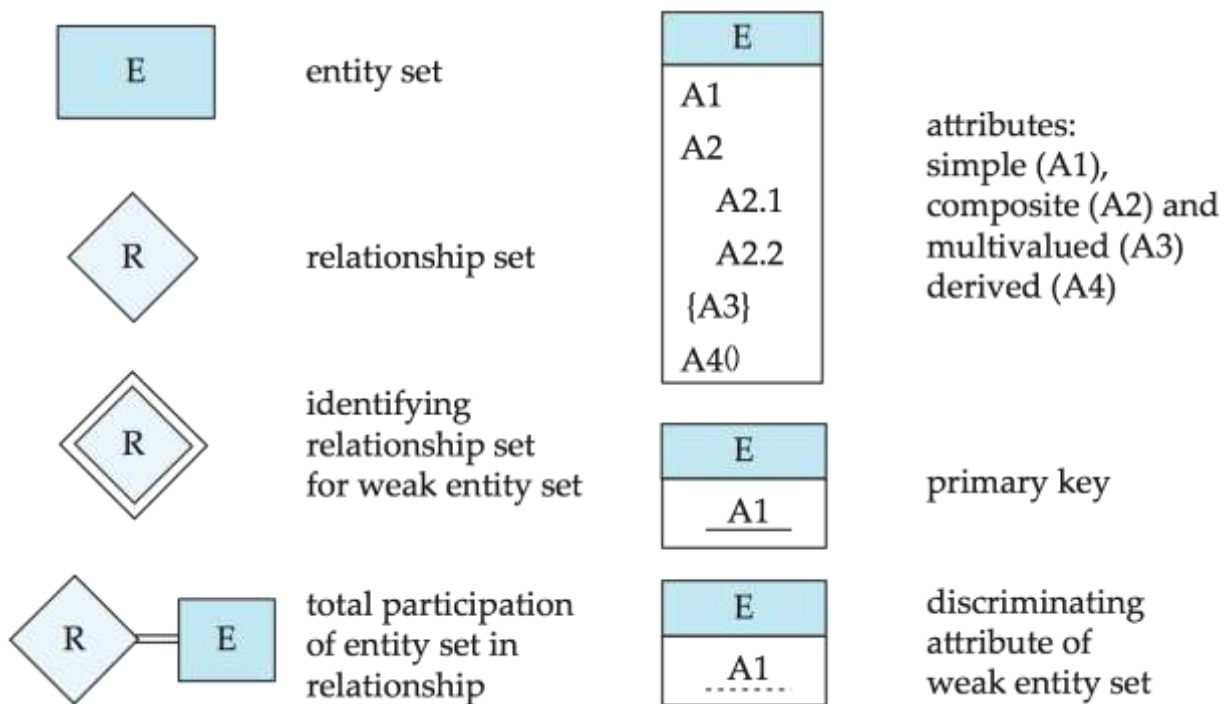
تؤدي إلى تكرار البيانات (Redundancy)، فإذا تم تعديل معلومة مشتركة مثل الاسم أو العنوان، يجب تعديلها في أكثر من مكان، مما قد يسبب مشاكل عدم التوافق بين البيانات.



■ ملاحظة تصميمية مهمة:

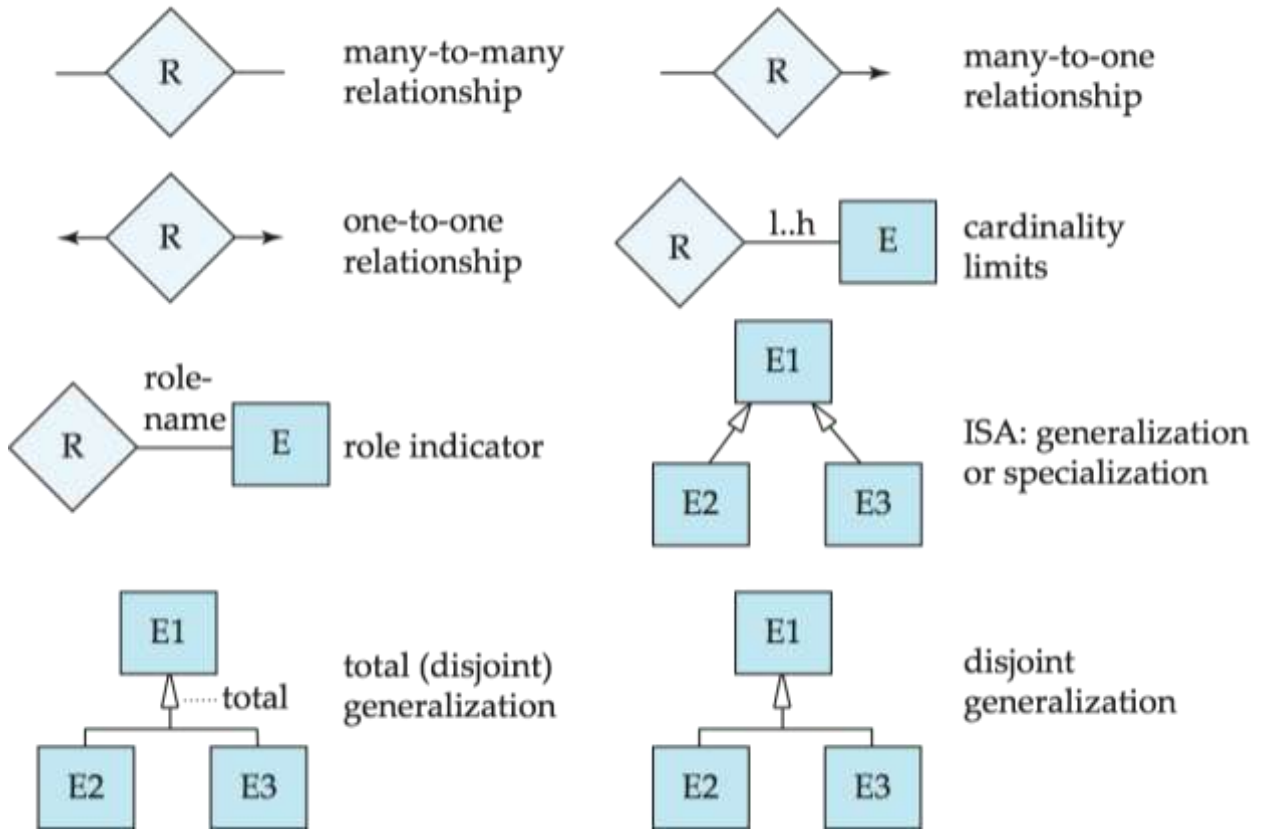
في قواعد المعطيات تكون عمليات القراءة (Read Operations) أكثر بكثير من عمليات الكتابة (Write Operations)، لذلك قد يتم أحياناً التضحية بجزء من الـ Normalization وتكرار بعض البيانات بهدف تحسين الأداء وتقليل عمليات الـ Join.

تذكير باصطلاحات التدوين:

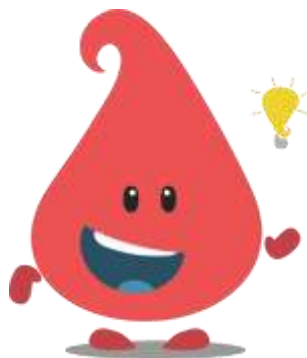


ملاحظات على اصطلاحات التدوين

- الكيان (Entity Set) يُمثل بمستطيل.
- الصفات (Attributes) تُكتب داخل مستطيل الكيان.
- الصفات المركبة تُزاح مستوياتها، متعددة القيم تُكتب داخل {}, والمشتقة تُكتب وبعدها قوسان ().
- العلاقة (Relationship Set) تُمثل بشكل مُعين (Diamond).
- العلاقة التعريفية الخاصة بالكيان الضعيف تُرسم بمُعين مزدوج.
- المشاركة الكلية (Total Participation) تُرسم بخط مزدوج.
- المفتاح الأساسي (Primary Key) يكون تحته خط متصل، والجزئي تحته خط متقطع.



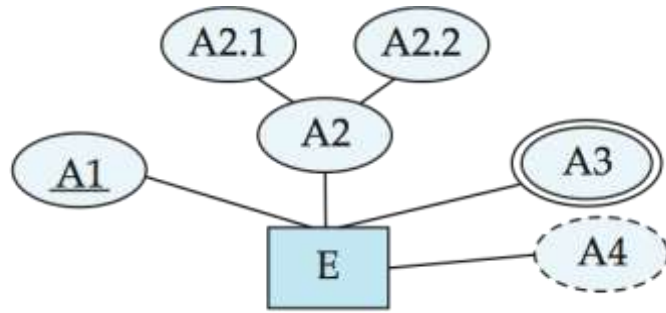
- حدود الكاردينالية (Cardinality Limits) تُستخدم لتحديد الحد الأدنى والحد الأعلى لعدد العناصر المشاركة في العلاقة .
- Role Name يُستخدم لتوضيح دور الكيان داخل العلاقة
- علاقة IS-A تعبر عن التخصيص أو التعميم بين الكيانات
- في حال لم يُكتب (total) عند العلاقة فهي partial.



اصطلاحات أخرى Chen

في الأمثلة السابقة كنا نستخدم اصطلاحات ER Notation الحديثة، أما هنا فنستخدم اصطلاحات Chen ، وهي من أقدم طرق تمثيل مخططات الكيانات والارتباطات.

entity set E with
simple attribute A1,
composite attribute A2,
multivalued attribute A3,
derived attribute A4,
and primary key A1



weak entity set



generalization



total
generalization



في اصطلاحات Chen تكون الواصفات (Attributes) مرسومة خارج الكيان ومرتبطة به، بعكس بعض الاصطلاحات الحديثة التي تضع الواصفات داخل الكيان نفسه.

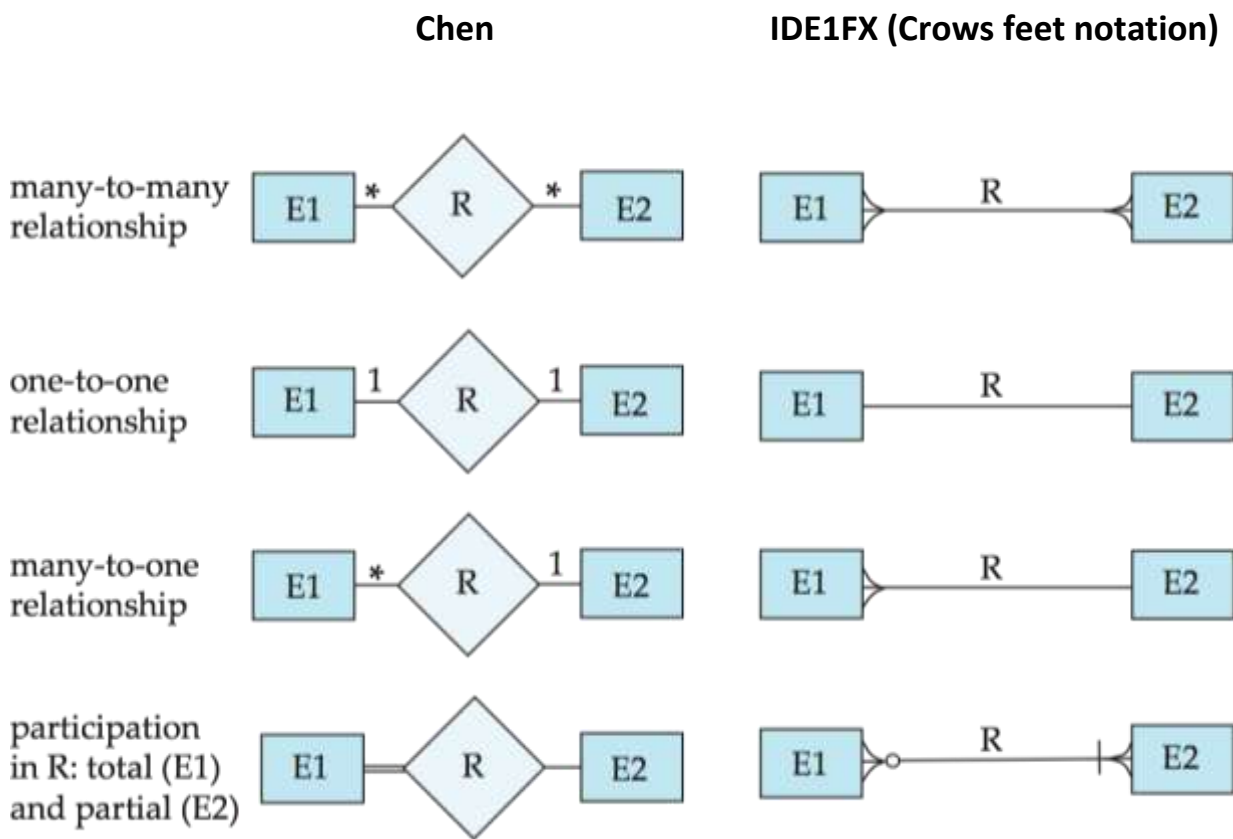
ملاحظات على الرسم:

- الواصفة البسيطة (Simple Attribute) تُمَثَّل بشكل بيضوي عادي.
- الواصفة المركبة (Composite Attribute) تتفرع منها واصفات فرعية.
- الواصفة متعددة القيم (Multivalued Attribute) تُمَثَّل بشكل بيضوي مزدوج.
- الواصفة المشتقة (Derived Attribute) تُمَثَّل بخط متقطع.
- المفتاح الأساسي (Primary Key) يكون تحته خط.
- الكيان الضعيف (Weak Entity) يُرسم بمستطيل مزدوج.
- في حالة ال Total Generalization يتم رسم خط مزدوج مع علاقة ISA.

اصطلاحات أخرى Chen, IDE1FX

مقارنة بين اصطلاحات Chen و IDE1FX (Crow's Foot Notation):

- تُستخدم عدة اصطلاحات مختلفة لتمثيل مخططات الكيانات والارتباطات، ومن أشهرها اصطلاحات Chen واصطلاحات Crow's Foot.
- يعتمد اصطلاح Chen على تمثيل العلاقات بشكل مُعَيَّن (Diamond)، بينما تعتمد اصطلاحات Crow's Foot على شكل الخطوط والأطراف لتوضيح نوع العلاقة والكاردينالية بشكل مختصر وأكثر استخدامًا في التطبيقات العملية.



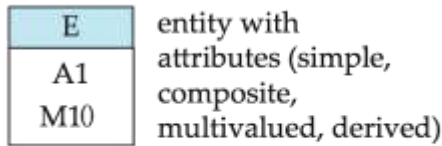
ملاحظات على الرسم:

- في اصطلاح Chen يتم تمثيل العلاقة بشكل مُعَيَّن بين الكيانيين.
- أما في Crow's Foot فيتم الاستغناء عن الشكل المُعَيَّن والاكتفاء بالخطوط والرموز.
- الرمز الذي يشبه قدم الغراب يدل على Many.
- الخط المفرد يدل على One.
- وجود دائرة صغيرة يدل على أن المشاركة اختيارية (Partial Participation).
- الخط العمودي يدل على مشاركة إجبارية (Total Participation).

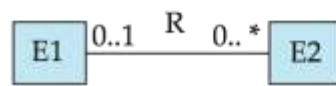
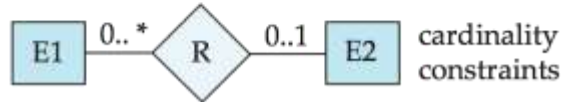
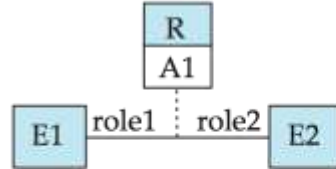
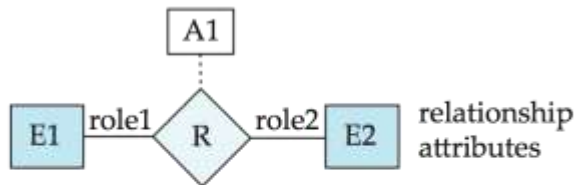
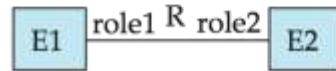
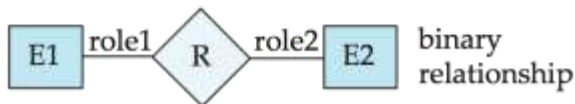
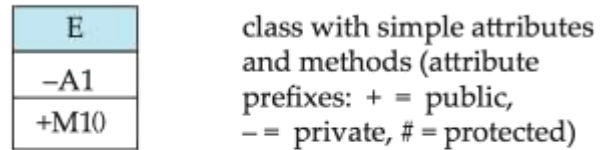
اصطلاحات أخرى UML

تعدّ UML اختصاراً لـ Unified Modeling Language ، تستخدم UML مخططات الصفوف (Class Diagrams)، والتي يمكن اعتبارها مكافئة لمخططات الكيانات والارتباطات (E-R Diagrams) ولكن بأسلوب أقرب إلى البرمجة كائنية التوجه.

ER Diagram Notation



Equivalent in UML

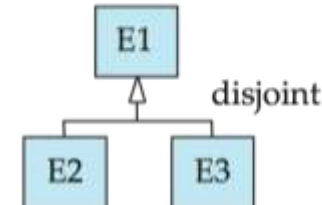
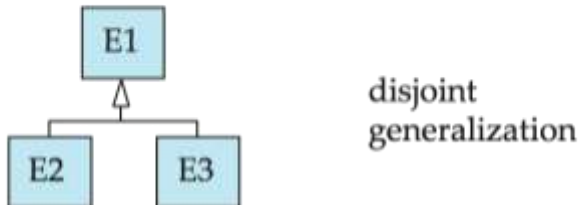
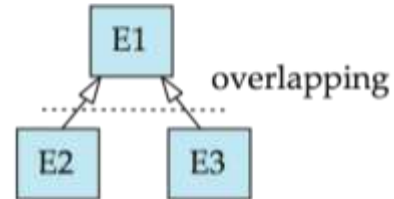
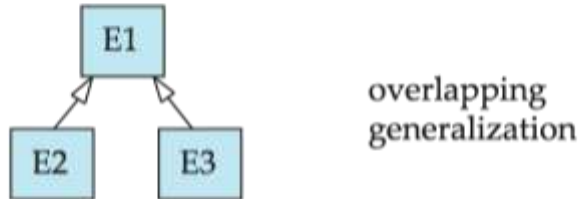
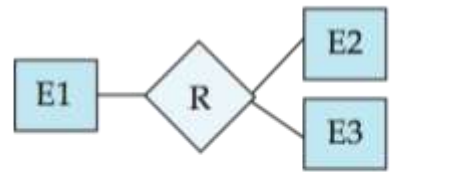
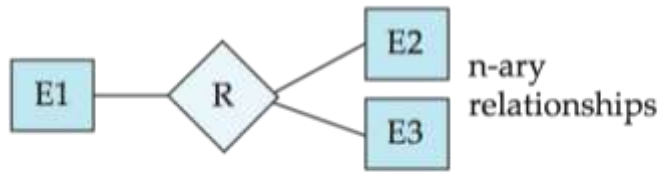


ملاحظات على الرسم:

- في UML يتم تمثيل الكيان على شكل Class يحتوي على الواصفات (Attributes) والعمليات (Methods).
- تُكتب الواصفات داخل الصندوق نفسه، وليس خارجه كما في اصطلاحات ER Notation.
- الرموز: (+) تعني Public \ (-) تعني Private \ (#) تعني Protected
- يتم تمثيل العلاقات بخطوط مباشرة بين الكيانات دون استخدام الشكل المُعيّن (Diamond).
- يمكن كتابة أسماء الأدوار (Role Names) على طرفي العلاقة لتوضيح دور كل كيان.
- إذا احتوت العلاقة على واصفات إضافية، يتم تمثيل العلاقة بصندوق مستقل يحوي الواصفات.
- يتم تمثيل الكاردينالية باستخدام القيم مثل: 0..1 \ 0..* \ وغيرها
- وذلك لتحديد عدد العناصر المشاركة في العلاقة، ولكن بعكس ما هي عليه في ER Notation.

ER Diagram Notation

Equivalent in UML



مدخل إلى لغة SQL

- تعتبر لغة SQL حاسوبية تستخدم للتعامل مع قواعد المعطيات العلاقية، ويعتمد أساسها النظري على الجبر العلاقائي (Relational Algebra).
- في الجانب النظري من المقرر لن يتم التركيز على نظام محدد مثل Oracle أو SQL Server ، وإنما سيتم شرح المفاهيم العامة المشتركة بين جميع أنظمة قواعد المعطيات، حيث يكون الاختلاف بينها غالباً في طريقة كتابة الأوامر (Syntax) وبعض التفاصيل الخاصة بكل نظام.
- تعتبر لغة SQL لغة تصريحية، حيث تركز على تحديد ما نريد تحقيقه دون الدخول في تفاصيل التنفيذ (What بدون How).

أنماط المجالات في SQL

1. char(n)

سلسلة نصية ثابتة الطول بطول محدد n .

2. varchar(n)

سلسلة نصية متغيرة الطول، بحيث يكون الحد الأقصى لطولها n .

♦ الفرق بين char(n) و varchar(n):

char(n) : يحجز دائماً عدداً ثابتاً من المحارف يساوي n ، حتى لو كانت القيمة المخزنة أقصر من ذلك.

varchar(n) : يكون الحد الأقصى لطوله n ، لكنه يحجز مساحة بحسب القيمة الفعلية المدخلة فقط.

✍ لذلك:

إذا كنا نعرف أن طول البيانات ثابت دائماً (كرقم الهاتف مثلاً)، فمن الأفضل استخدام char، أما إذا كان الطول متغيراً فمن الأفضل استخدام varchar لتوفير المساحة.

3. int

عدد صحيح.

4. smallint

عدد صحيح صغير الحجم.

5. numeric(p,d)

عدد عشري ثابت، حيث : p تمثل عدد الخانات الكلي - d تمثل عدد الخانات بعد الفاصلة.

(وهناك أنظمة تعتبر p تمثل عدد الخانات الموجودة قبل الفاصلة)

✍ مثال:

numeric(5,2)

يعني عدداً مكوناً من 5 خانة، منها خانتان بعد الفاصلة.

6. float(n)

عدد عشري تقريبي يحتوي على دقة لا تقل عن n خانة.

7. double precision و real

أنواع تستخدم لتخزين الأعداد العشرية التقريبية، مع دقة أعلى في حالة double precision.

بناء الجداول :

- مجموعة من الواصفات $A_1, A_2, \dots, A_n$
- مجالات الواصفات $D_1, D_2, \dots, D_n$ $A_1, A_2, \dots, A_n$
- بنية علاقاتية $R(A_1, A_2, \dots, A_n)$

مثلاً:

- instructor (ID, name, dept_name, salary)
- create table r ($A_1 D_1, A_2 D_2, \dots, A_n D_n$,
(integrity-constraint₁),
... ,
(integrity-constraint_k)
);
- create table instructor (
ID char(5),
name varchar(20) not null,
dept_name varchar(20),
salary numeric(8,2)
);

فيتم بناء جداول فارغة بتعليمة create وبعدها اسم الجدول، ثم كل واصفة مع الـ datatype الخاص بها، ثم القيود.

القيود

- not null
- primary key ($A_1, \dots, A_n$) (not null يتضمن)
- foreign key ($A_m, \dots, A_n$) references r
- create table instructor (
ID char(5),
name varchar(20) not null,
dept_name varchar(20),
salary numeric(8,2),
primary key (ID),
foreign key (dept_name) references department)

قيود إضافية :

- UNIQUE

يُستخدم لمنع تكرار القيم داخل الحقل.

UNIQUE(A1, A2)

يمكن تطبيق القيد على أكثر من حقل معاً، بحيث لا تتكرر نفس التركيبة بين الحقول.

- CHECK

يُستخدم لفرض شرط معيّن على القيم المدخلة.



تعديل الجداول وحذفها :

- alter table

alter table r add A D

تستخدم لإضافة عمود جديد إلى جدول موجود مسبقاً.

حيث:

(r اسم الجدول - A اسم العمود الجديد - D نوع البيانات)

ملاحظة:

الأسطر القديمة تأخذ null في العمود الجديد.

alter table r modify A D

تستخدم لتعديل خصائص عمود موجود مسبقاً، مثل تغيير نوع البيانات أو حجمها أو بعض القيود (مثل NULL و NOT NULL).

ملاحظة أربسية:

- إضافة أو حذف المفاتيح الأساسية (Primary Key) والمفاتيح الخارجية (Foreign Key) تتم باستخدام أوامر أخرى ضمن ALTER TABLE مثل ADD CONSTRAINT و DROP CONSTRAINT.

alter table r drop A

تستخدم لحذف عمود من جدول، وليست موجودة في جميع الأنظمة

- drop table

تستخدم لحذف جدول بأكمله، وفي حال وجود بيانات به يتم تلقي تحذير بذلك.

```

■ create table student (
    ID          varchar(5) primary key,
    name        varchar(20) not null,
    dept_name   varchar(20),
    tot_cred    numeric(3,0),
    foreign key (dept_name) references department);

```

نلاحظ أنه يمكن وضع عبارة الـ Primary key على نفس مستوى الـ ID لأنهما جزء واحد.

```

■ create table course (
    course_id   varchar(8) primary key,
    title       varchar(50),
    dept_name   varchar(20),
    credits     numeric(2,0),
    foreign key (dept_name) references department);

```

```

■
create table takes (
    ID          varchar(5),
    course_id   varchar(8),
    sec_id      varchar(8),
    semester    varchar(6),
    year        numeric(4,0),
    grade       varchar(2),
    primary key (ID, course_id, sec_id, semester, year)
    foreign key (ID) references student,
    foreign key (course_id, sec_id, semester, year) references section
);

```

ملاحظة: الـ PowerDesigner برنامج لتصميم وتحليل قواعد البيانات، يمكن من خلاله التحويل من Conceptual Model إلى Physical Model، وتوليد تعليمات الـ SQL لإنشاء الجداول من الـ Physical Model.



الاستعلام :

■ select $A_1, A_2, \dots, A_n$ From $r_1, r_2, \dots, r_m$ where P

- هي أكثر تعليمة استخداماً في SQL تُستخدم لإنشاء الاستعلامات لاستدعاء البيانات (الأعمدة) من الجداول.
- نتيجة الاستعلام علاقة جديدة.

مثال:

select name from instructor where dept_name='Comp. Sci.';

سيحضر أسماء الأساتذة ذوي اختصاص computer science
 أسماء الجداول الشرط
 الأعمدة

- يمكن كتابة أوامر SQL باستخدام أحرف صغيرة (lower case) أو أحرف كبيرة (upper case)، لكن من الأفضل توحيد طريقة الكتابة دائماً، لأن ذلك يساعد نظام الـ Cache على التعرف على الاستعلامات المتشابهة وتنفيذها بشكل أسرع، فعند تنفيذ كل استعلام يتم ترجمة التعليمة (Compilation) وتخزين خطوات تنفيذها في Cache خاص بالتعليقات، ويتم إحضار المعلومات المطلوبة وتخزينها في Cache خاص بالمعلومات التي تمت قراءتها من القرص الصلب، ثم في حال إعادة تنفيذ التعليمة المكتوبة بنفس نمط الأحرف سيتم التعرف عليها بدون عمل Compilation، وسيتم قراءة المعلومات مباشرة من الـ RAM، أما في حال كتابتها بنمط أحرف مختلف فسيعتبرها النظام تعليمة جديدة، وبالتالي سيقوم بإعادة عمل Compilation والممرور بكافة خطوات تنفيذها، والذهاب إلى القرص الصلب لإحضار المعلومات، مما سيستهلك وقتاً.
- عند عدم وجود شرط يتم إرجاع جميع الأسطر من الجدول، ويكافئ ذلك عملية الإسقاط في الجبر العلاقتي.
- Select * from instructor
 الرمز * يُستخدم لإرجاع جميع الواصفات (الأعمدة) الموجودة في الجدول.
- Select ID, name, salary/12 from instructor
 يمكن استخدام عمليات رياضية داخل الاستعلام، سواء على الثوابت أو على الواصفات، ويمكن عمل ذلك أيضاً في الجبر العلاقتي.
- مثال: salary/12 لحساب الراتب الشهري انطلاقاً من الراتب السنوي.

ملاحظة: التكرار مسموح بشكل افتراضي في نتائج SQL ، بعكس الجبر العلاقتي.

الكلمة DISTINCT :

select distinct dept_name from instructor

تُستخدم لإزالة القيم المكررة من ناتج الاستعلام، ويمكن تطبيقها على أكثر من حقل معاً، بحيث يتم حذف التكرار اعتماداً على تركيبة القيم بين هذه الحقول.

select all dept_name from instructor

لا تغيّر سلوك SQL فعلياً، لأن SQL يعيد القيم المكررة بشكل افتراضي، وإنما تُستخدم للتوضيح فقط، بحيث يفهم المبرمج أن المطلوب هو الإبقاء على التكرار وعدم حذفه.

الشروط :

select name from instructor

where dept_name = 'Comp. Sci.' and salary > 80000

- تكافئ عملية الاختيار select في الجبر العلاقتي
 - يمكن استخدام الروابط المنطقية and, or, not
 - يمكن استخدام between للتعبير عن المجالات
 - مثال : where salary between 90000 and 100000 يعطي جميع الرواتب المحصورة بين 90000 و100000 بما فيها أطراف المجال (90000 و100000)
 - يمكن مقارنة قيم متعددة أو أسطر كاملة
- Select name , course_id From instructor , teaches
where (instructor.ID,dept_name) = (teaches.ID,'Biology');
ويعني ذلك :
Instructor.ID = teaches.ID and dept_name = 'Biology'
أي اختيار المدرّسين المرتبطين بجدول teaches والمنتتمين إلى قسم Biology

الجدول المعنية :

- وجود أكثر من جدول بدون شروط يعني جداء ديكارتي
- الجداء الديكارتي غير مفيد بدون شروط أخرى لأن الناتج يصبح ضخماً ويحتوي تركيبات غير منطقية غالباً.

Select * from instructor , teaches

ID	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	60000

ID	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009

Inst.ID	name	dept_name	salary	teaches.ID	course_id	sec_id	semester	year
10101	Srinivasan	Physics	95000	10101	CS-101	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	CS-315	1	Spring	2010
10101	Srinivasan	Physics	95000	10101	CS-347	1	Fall	2009
10101	Srinivasan	Physics	95000	10101	FIN-201	1	Spring	2010
10101	Srinivasan	Physics	95000	15151	MU-199	1	Spring	2010
10101	Srinivasan	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...
12121	Wu	Physics	95000	10101	CS-101	1	Fall	2009
12121	Wu	Physics	95000	10101	CS-315	1	Spring	2010
12121	Wu	Physics	95000	10101	CS-347	1	Fall	2009
12121	Wu	Physics	95000	10101	FIN-201	1	Spring	2010
12121	Wu	Physics	95000	15151	MU-199	1	Spring	2010
12121	Wu	Physics	95000	22222	PHY-101	1	Fall	2009
...	...	...	...	...	...	...	...	...
...	...	...	...	...	...	...	...	...

الضم Join:

جداء ديكارتي لكن مع شروط

أوجد أسماء المدرسين الذين أعطوا دروساً مع أرقام المواد التي أعطوها

```
select name, course_id
from instructor, teaches
where instructor.ID = teaches.ID
```

أوجد رقم المادة وعنوانها والفصل والسنة للمواد التي يعطيها قسم النظم المعلوماتية

```
select section.course_id, title, semester, year
from section, course
where section.course_id = course.course_id and dept_name = 'Comp. Sci.'
```



الضم الطبيعي (Natural Join)

- يُستخدم ال Natural Join لربط الجداول اعتماداً على الواصفات المشتركة التي تحمل الاسم نفسه والقيم نفسها.
- عند تنفيذ ال Natural Join يتم:
 - مطابقة الأسطر التي تمتلك القيم نفسها في الواصفات المشتركة.
 - إظهار نسخة واحدة فقط من الواصفات المشتركة في الناتج.

مثال:

```
select * from instructor natural join teaches
```

يقوم هذا الاستعلام بربط الجدولين اعتماداً على الواصفات المشتركة بينهما.

يجب الانتباه جيداً إلى جميع الواصفات التي تحمل الاسم نفسه بين الجداول، لأن ال Natural Join يستخدمها تلقائياً في عملية الربط.

في المثال التالي:

```
select name, title
```

```
from instructor natural join teaches natural join course;
```

سيتم الربط ليس فقط باستخدام: course_id ، بل أيضاً باستخدام: dept_name

لأنها واصفة مشتركة بين: instructor , course

وبالتالي سيظهر فقط المدرّسون الذين قاموا بتدريس مواد ضمن نفس القسم الذي ينتمون إليه، وهذا ليس المطلوب - > لذلك يُعتبر هذا الاستعلام خطئاً منطقياً.

التصحيح يكون بتحديد شرط الربط بشكل صريح:

```
select name, title
```

```
from instructor natural join teaches, course
```

```
where teaches.course_id = course.course_id
```

ويمكن أيضاً كتابة التصحيح بطريقة أخرى باستخدام : using(course_id)

■ مثال:

```
select name, title
```

```
from (instructor natural join teaches)
```

```
join course using(course_id);
```

- بحيث يتم الربط فقط اعتماداً على: course_id دون استخدام الواصفات المشتركة الأخرى تلقائياً.

إعادة التسمية :

يمكن في SQL إعادة تسمية الواصفات Attributes أو العلاقات Tables باستخدام الكلمة AS بهدف جعل أسماء الناتج أو الاستعلام أوضح وأسهل للقراءة، وتستخدم على مستوى العمود أو الجدول.

مثال:

```
select ID, name, salary/12 as monthly_salary
from instructor
```

هنا استخدمنا as لتفادي ظهور اسم العمود salary/12.

الكلمة as يمكن إغفالها " ليست إجبارية في SQL عند إعادة التسمية، ويمكن حذفها مع بقاء المعنى نفسه. "

■ أوجد أسماء المدرسين الذين يقبضون راتباً أعلى من أحد مدرسي قسم النظم المعلوماتية

```
select distinct T. name
from instructor as T, instructor as S
where T.salary > S.salary and S.dept_name = 'Comp. Sci.'
```

ملاحظة: تم إعطاء اسم مختصر للجدول وذلك لاستخدام الجدول نفسه أكثر من مرة داخل الاستعلام، فقارنا بين رواتب النسخة الأولى من الجدول ورواتب النسخة الثانية (مدرسي قسم المعلوماتية فقط).

ترتيب النتائج (Order By):

1. تُستخدم العبارة order by لترتيب ناتج الاستعلام بحسب واصفة واحدة أو عدة واصفات.

مثال:

```
select distinct name from instructor order by name
```

- يقوم هذا الاستعلام بترتيب الأسماء تصاعدياً حسب الوصفة name .

2. يمكن الترتيب بحسب أكثر من واصفة.

مثال:

```
order by dept_name, name
```

- هنا يتم أولاً الترتيب حسب dept_name ، وعند تساوي القيم فيه يتم الاعتماد على name لإكمال الترتيب داخل كل قسم.

3. يمكن أن يكون الترتيب:

- تصاعدياً باستخدام asc
- أو تنازلياً باستخدام desc

مثال:

order by name desc

- أي ترتيب الأسماء ترتيباً تنازلياً.

ملاحظة : يمكن أن يكون الترتيب رقمياً أو أبجدياً بحسب نوع الوصفة المستخدمة في الترتيب.

أمثلة :

Symbol (Name)	Example of Use
σ (Selection)	$\sigma_{\text{salary} \geq 85000}(\text{instructor})$ Return rows of the input relation that satisfy the predicate.
Π (Projection)	$\Pi_{\text{id}, \text{salary}}(\text{instructor})$ Output specified attributes from all rows of the input relation. Remove duplicate tuples from the output.
$\bowtie$ (Natural Join)	$\text{instructor} \bowtie \text{department}$ Output pairs of rows from the two input relations that have the same value on all attributes that have the same name.
$\times$ (Cartesian Product)	$\text{instructor} \times \text{department}$ Output all pairs of rows from the two input relations (regardless of whether or not they have the same values on common attributes)
$\cup$ (Union)	$\Pi_{\text{name}}(\text{instructor}) \cup \Pi_{\text{name}}(\text{student})$ Output the union of tuples from the two input relations.

انتهى !!